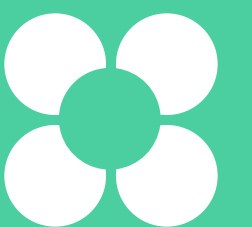


Пакеты и модули в Go

Матвей Ефимов
Эксперт в Go, Python, SQL



Матвей Ефимов

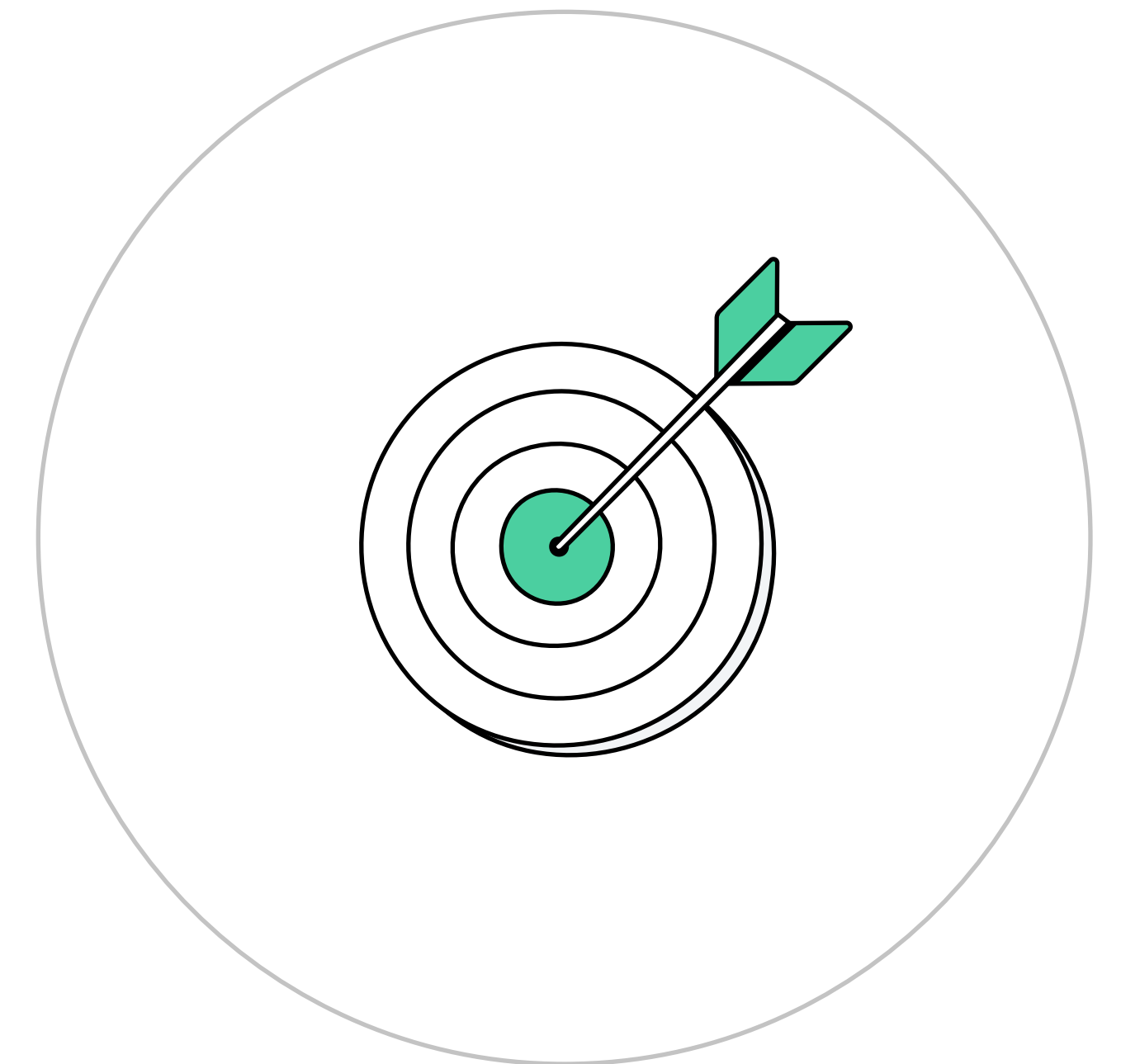
О спикере:

- Преподаватель с более чем 5-летним опытом в IT-образовании
- Разработчик учебных программ и эксперт в Go, Python, SQL
- Фриланс-программист и наставник студентов от школ до вузов



Цели занятия

- Понять, зачем нужны пакеты и модули в Go
- Разобраться, как устроен Go-проект и как организовать структуру кода
- Понимать принцип создания и подключения собственных пакетов
- Узнать, как работает модульность и управление зависимостями через `go.mod`
- Сравнить подход Go с другими языками программирования



План занятия

- 1 Зачем нужны пакеты и модули в Go
- 2 Что такое пакет (package) в Go
- 3 Что такое модуль (module) в Go
- 4 Структура проекта на Go
- 5 Подключение и использование своих пакетов
- 6 Чем отличается подход Go от других языков
- 7 Итоги



Зачем нужны пакеты и модули в Go

Зачем нужны пакеты и модули в Go

Представьте, что вы строите дом. Один человек не справится со всем: нужен электрик, сантехник, каменщик

Так же и в программировании — весь код в одном файле неудобен и непрактичен

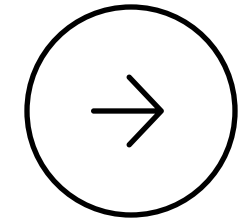
Модульность позволяет делить код на части, отвечающие за свои задачи

Как это реализовано в Go

Go — минималистичный язык без классов и объектной модели. Вместо этого используется простая структура:

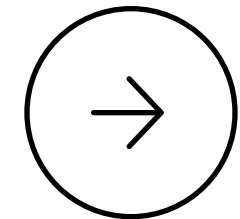
- Код группируется в пакеты
- Зависимости и версии управляются через модули

Как это реализовано в Go



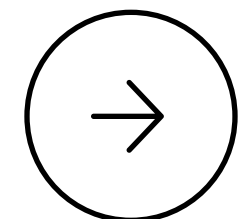
Делить код на логические части

Упрощают навигацию, позволяют выделить вспомогательные функции, обработчики и т.п.



Переиспользовать код в разных проектах

Один и тот же пакет можно подключить в других местах без копипаста



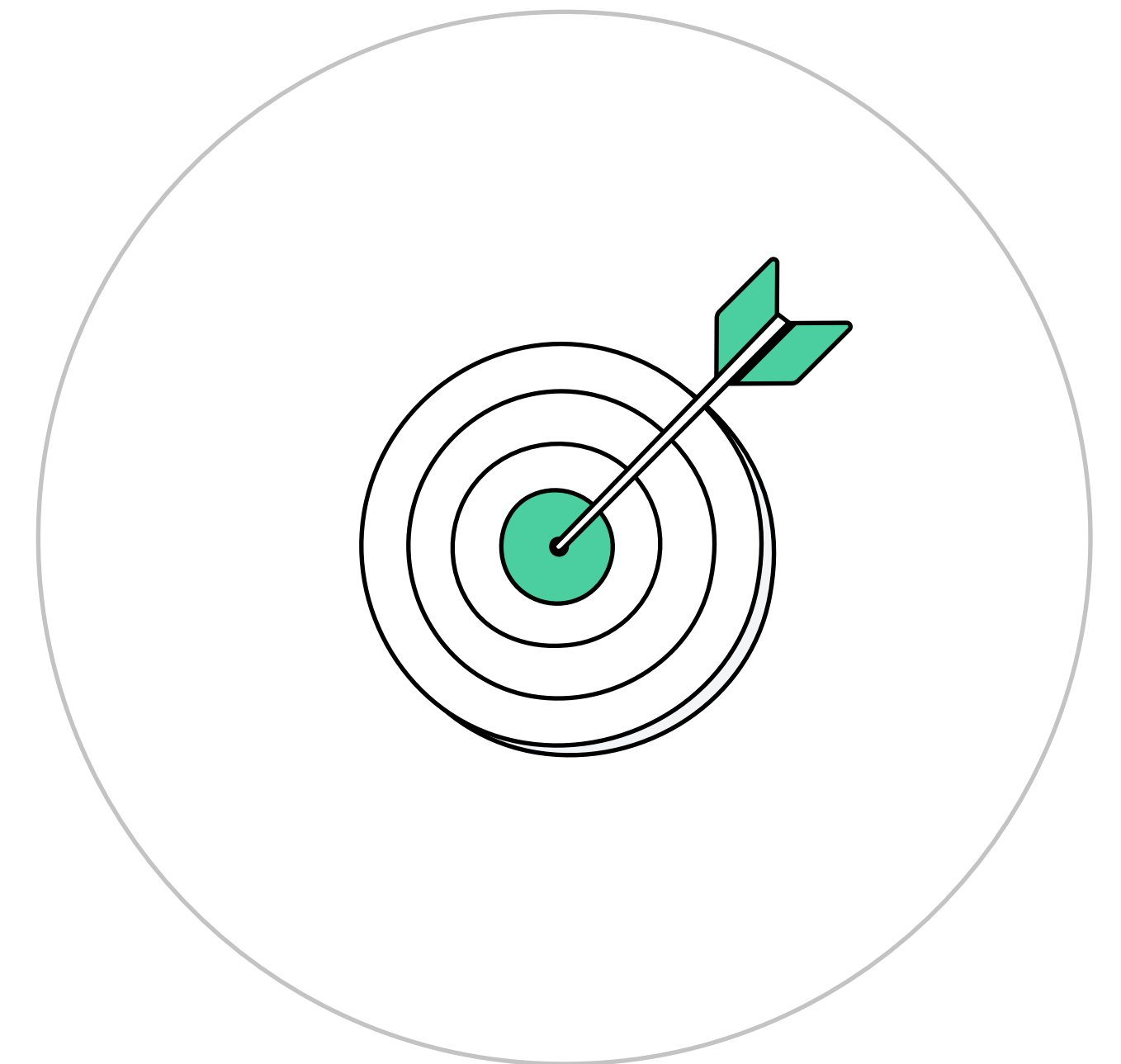
Избегать конфликтов имён и упрощать сопровождение

Функции и переменные в разных пакетах не мешают друг другу

Что такое пакет (package) в Go

Цели видео

- Разобраться, что такое пакет в Go и зачем он нужен
- Узнать, как объявляется пакет в .go-файле
- Понять роль пакета `main` и функции `main()`
- Увидеть, как подключать и использовать функции из другого пакета



Что такое пакет в Go

Пакет — основная единица организации кода в Go

- Он группирует функции, структуры, интерфейсы и переменные
- Каждый .go-файл должен принадлежать какому-то пакету

```
package main
```


Пакет `main` и функция `main`

Если пишете запускаемое приложение:

- используйте пакет `main`
- добавьте точку входа — функцию `main()`

```
package main

func main() {
    // точка входа в программу
}
```

Пакеты для библиотек и утилит

**Если пишете библиотеку
или вспомогательный код:**

- используйте другое имя пакета, например `mathutils`
- такой код нельзя запустить сам по себе, но его можно импортировать

```
package mathutils
```

Пример: создаём свой пакет

Создаём файл mathutils.go с функцией сложения:

```
package mathutils

func Add(a int, b int) int {
    return a + b
}
```


Импорт и использование пользовательского пакета

Импортируем пакет в другой части проекта
и используем функцию:

```
import "myproject/mathutils"  
result := mathutils.Add(2, 3)
```

Стандартные пакеты Go

Go поставляется с богатой стандартной библиотекой:

- `fmt` — вывод текста
- `os` — работа с файловой системой
- `math` — математические функции
- `time` — работа со временем
- `strings` — обработка строк

```
import "fmt"
```


Итоги видео

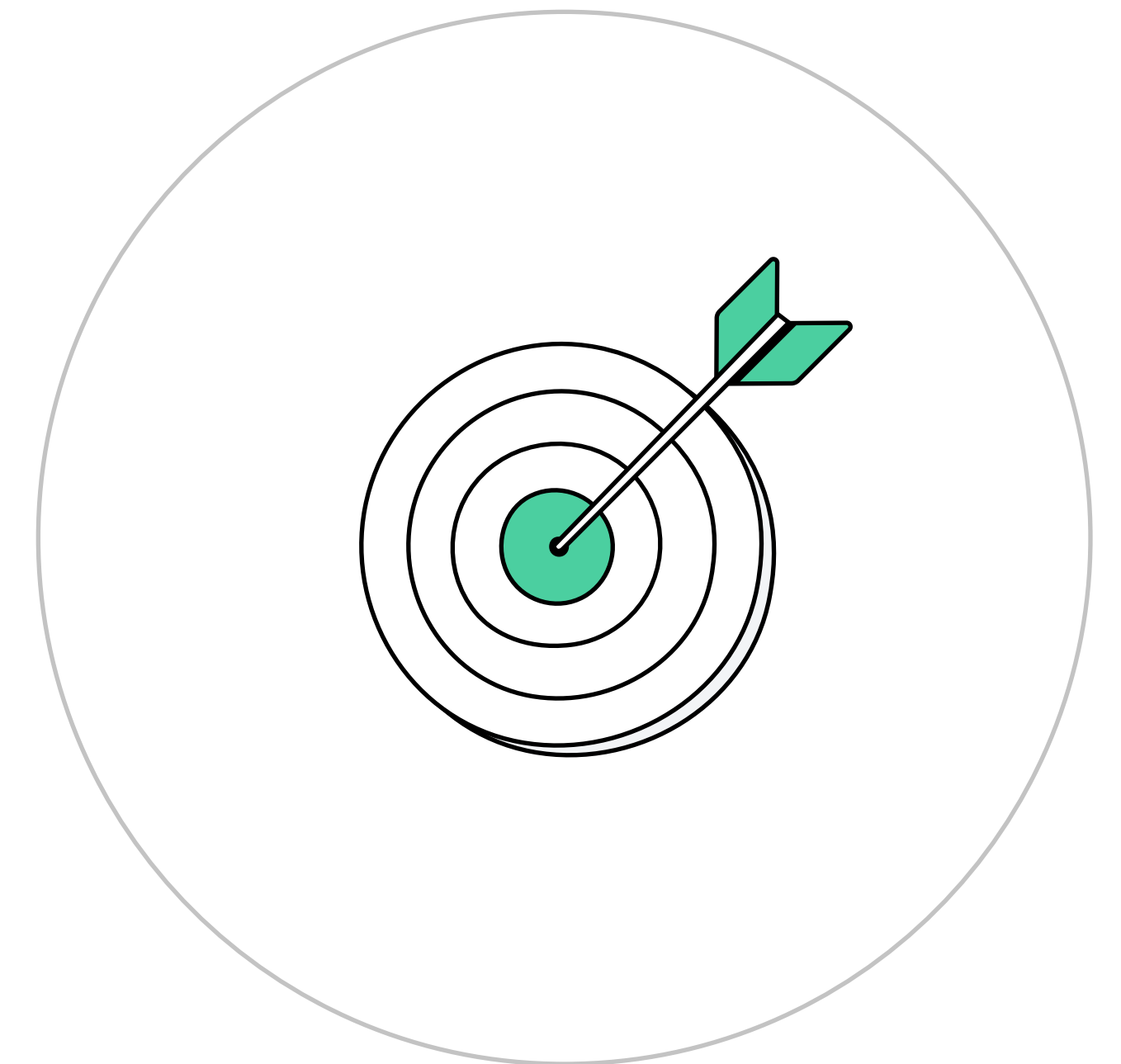
- Каждый .go-файл должен принадлежать какому-то пакету
- Пакеты группируют функции, структуры и интерфейсы по смыслу
- Пакет main используется для запуска приложений и содержит функцию main()
- Пользовательские пакеты создаются в отдельных папках с объявлением package имя
- Чтобы использовать пакет в другом файле, его нужно импортировать по пути: имя_модуля/путь_к_пакету



Что такое модуль (module) в Go

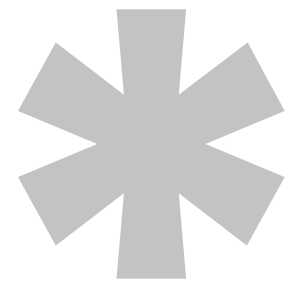
Цели видео

- Понять, что такое модуль в языке Go и зачем он нужен
- Разобраться, как модули помогают управлять проектом, зависимостями и сборкой
- Научиться инициализировать модуль с помощью команды `go mod init`
- Увидеть, как устроен файл `go.mod` и какую информацию он содержит
- Освоить базовые принципы организации Go-проекта через модули



Зачем нужен модуль в Go

- Когда в проекте много пакетов и библиотек, нужен способ всё связать
- Модуль помогает управлять проектом как целым
- Он объединяет пакеты, зависимости и настройку сборки



Что такое модуль

Модуль — это единица проекта в Go

Он содержит один или несколько связанных пакетов

Модуль = проект = библиотека

Что даёт модуль

Модуль позволяет Go:

- понять структуру проекта
- подключать внешние библиотеки
- управлять версиями
- обеспечить повторяемость сборки

Как создать модуль

Перейдите в папку проекта и выполните:

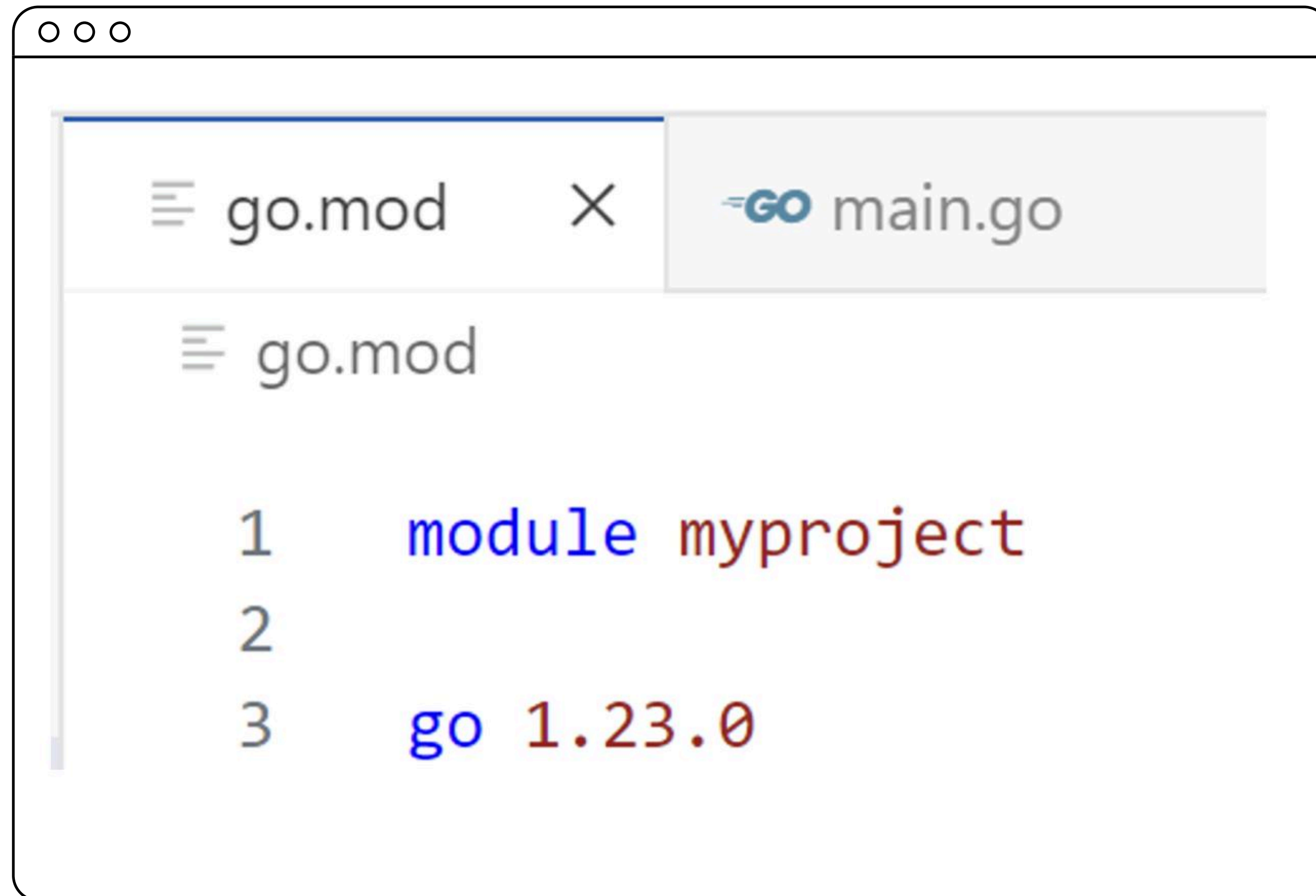
```
go mod init myproject
```

- myproject — имя модуля
- может быть путём до репозитория:
github.com/user/myproject

После этого создаётся файл go.mod

Пример: структура проекта

Файл go.mod создаётся автоматически:



The image shows a code editor window with a tab labeled 'go.mod' and a 'GO' logo. The editor displays the following Go module declaration:

```
1  module myproject
2
3  go 1.23.0
```


Что хранит go.mod

→ **Файл go.mod — это «паспорт» проекта**

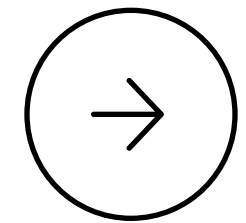
→ **Он фиксирует:**

- имя модуля
- версию Go
- список подключённых библиотек и их версии

Почему это удобно

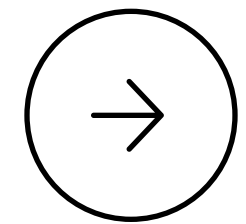
- Упрощает подключение зависимостей
- Go сам управляет версиями
- Не нужно вручную настраивать сборку
- Легко воспроизвести проект на любом компьютере

Почему без go.mod проект не запустится



Go требует go.mod для:

- подключения сторонних библиотек
- создания полноценного проекта



Даже маленький проект — это модуль

Итоги видео

- Модуль объединяет все пакеты проекта и делает его целостным
- Для работы с модулями используется файл `go.mod`, создаваемый через `gomodinit`
- В `go.mod` фиксируются имя проекта, версия Go и зависимости
- Модули обеспечивают повторяемость сборки и удобное подключение внешних библиотек
- Каждый полноценный Go-проект должен быть модулем — даже самый простой

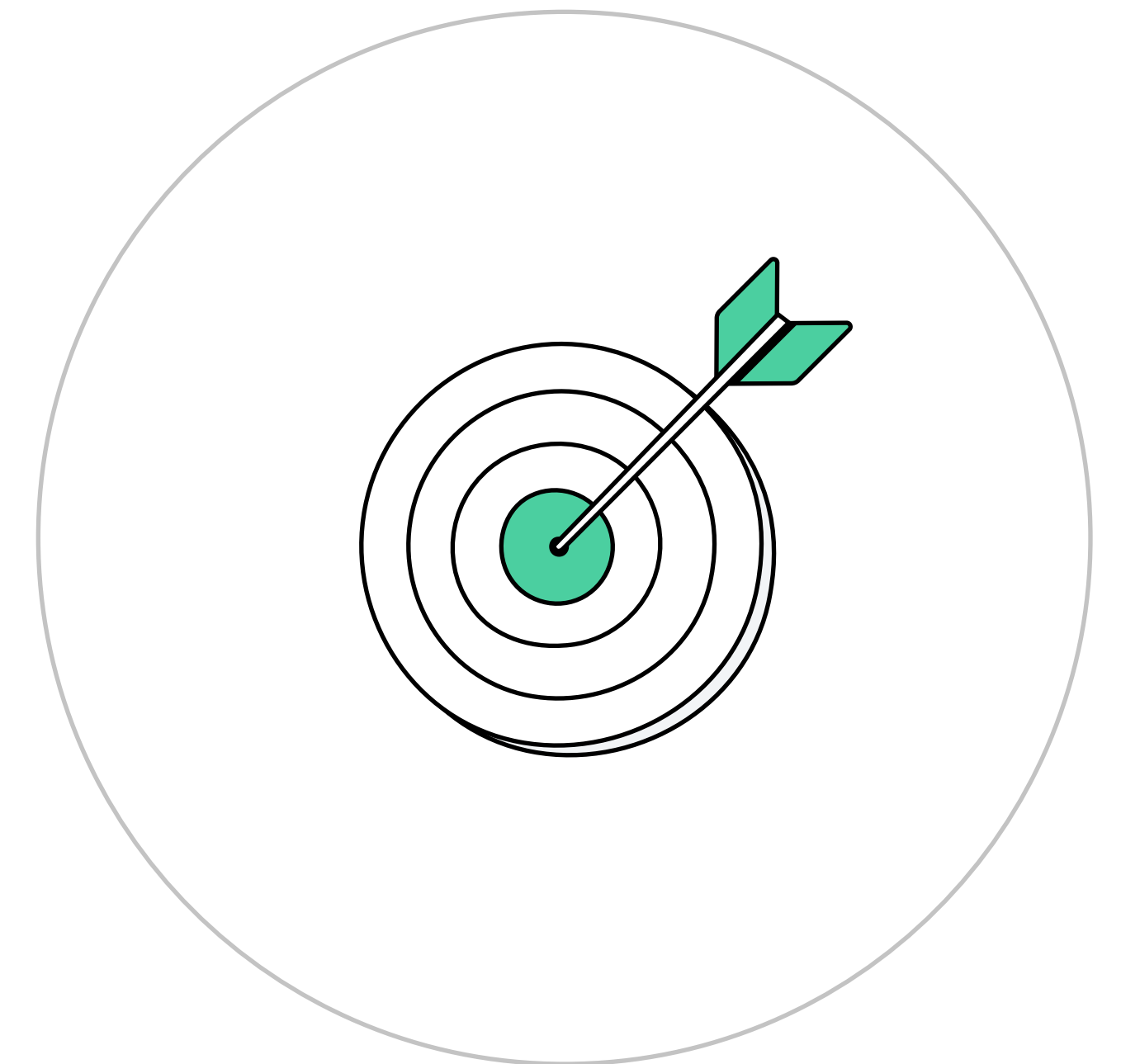


Структура проекта на Go



Цели видео

- Понять, как устроена структура проекта на Go
- Узнать, где размещаются файлы `main.go` и `go.mod`
- Разобраться, как пакеты и папки соотносятся между собой
- Увидеть, как организовать вспомогательный код в отдельных папках
- Научиться читать и создавать минимальную структуру Go-проекта



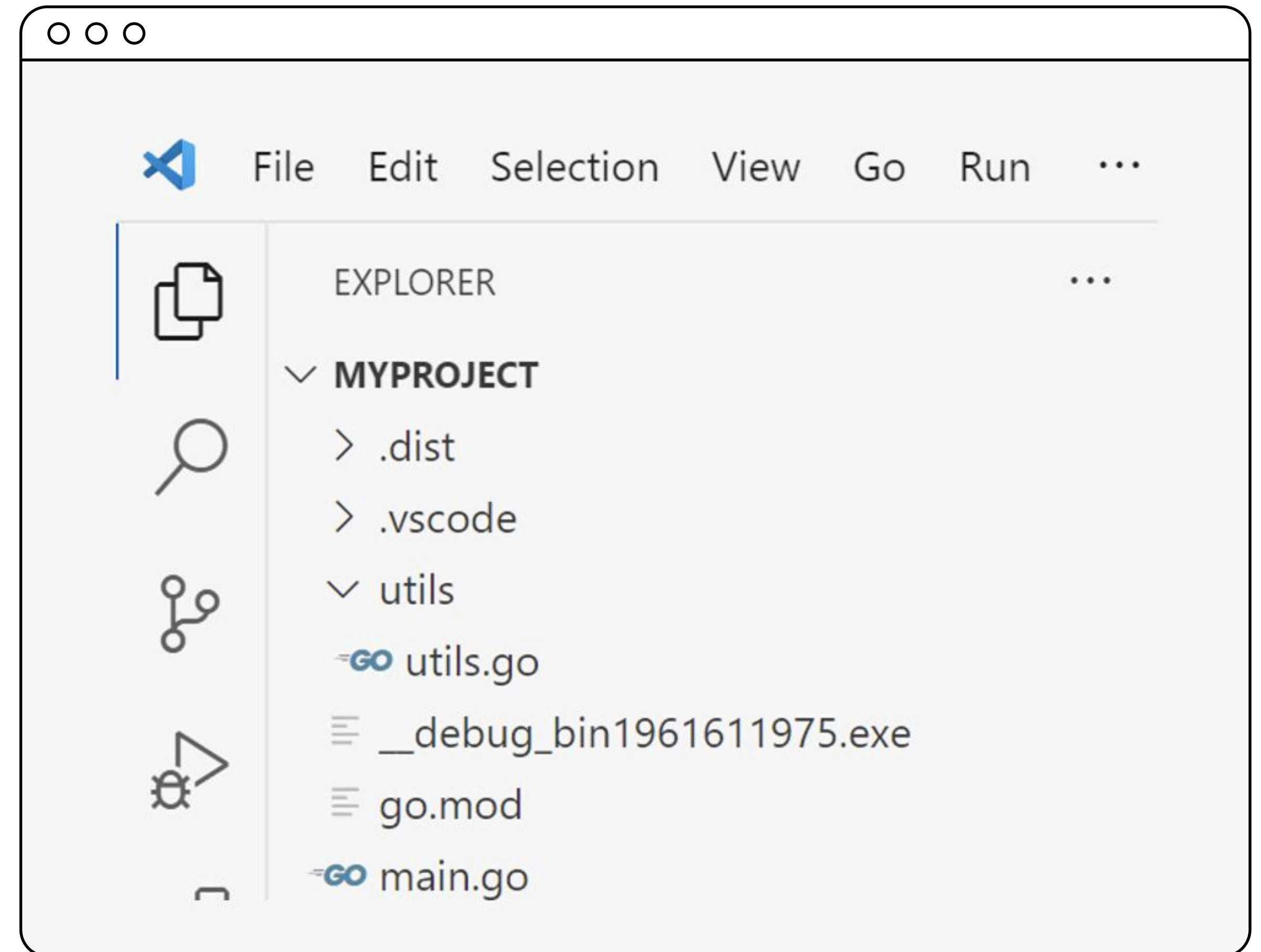
Что такое структура проекта в Go

- Каждый Go-проект — это модуль, состоящий из пакетов
- Обычно один пакет = одна папка с .go-файлами
- В корне проекта — файл go.mod

Пример структуры проекта

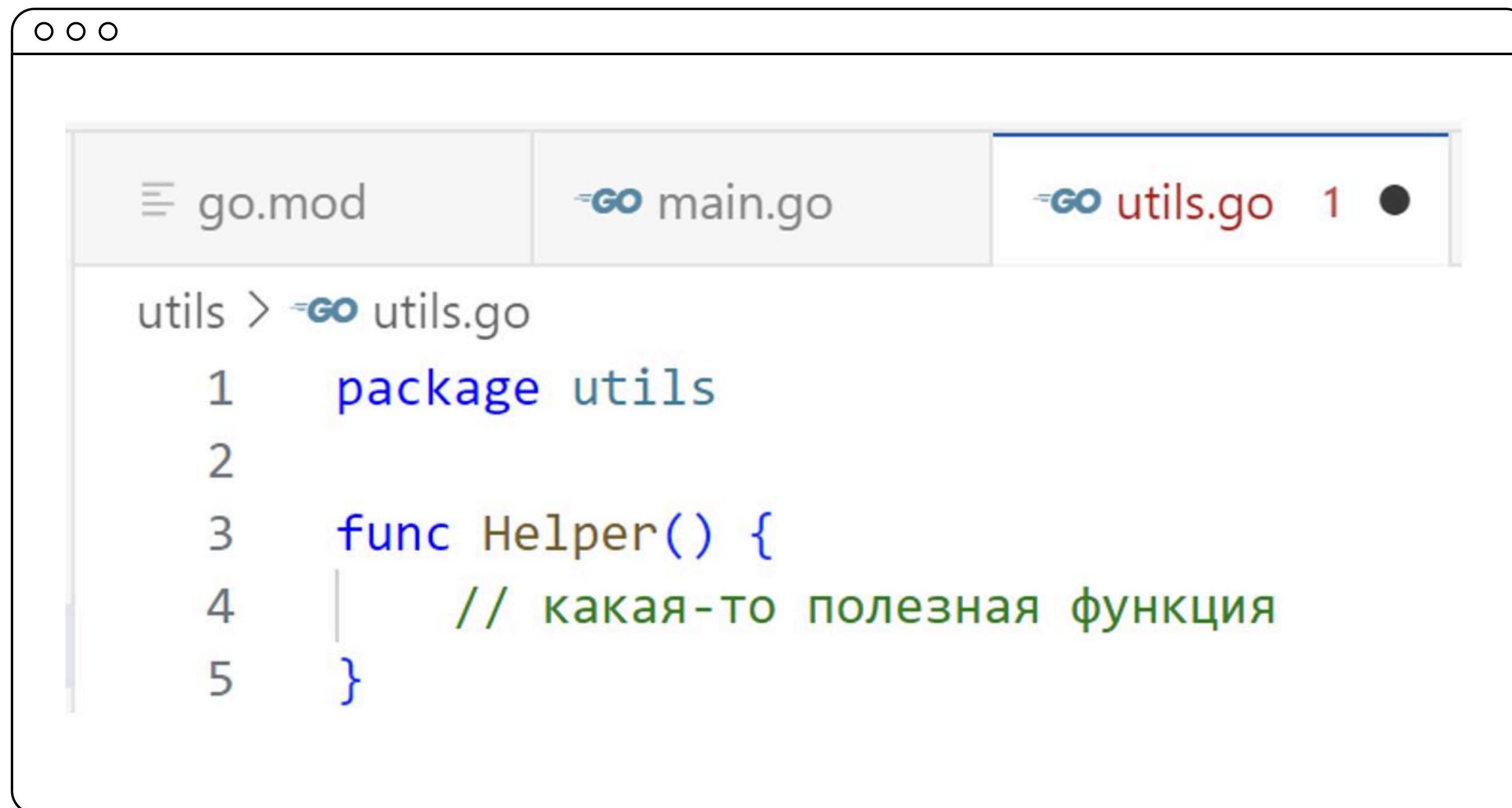
Проект myproject

- go.mod – описывает модуль
- main.go – точка входа
- utils/ – папка с утилитами



Код в файле utils.go

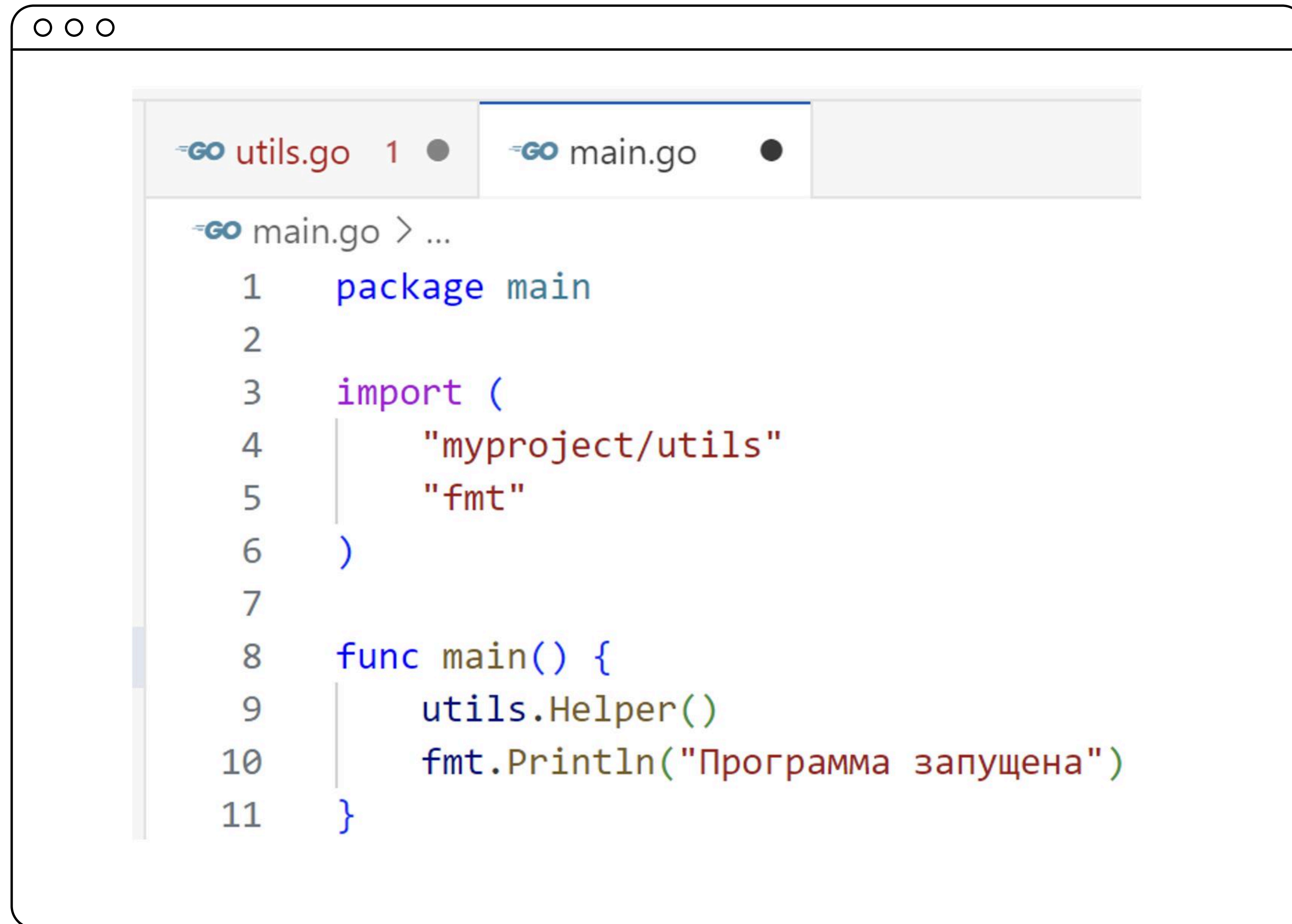
Этот файл принадлежит пакету utils



The screenshot shows a Go IDE window with three tabs: 'go.mod', 'main.go', and 'utils.go'. The 'utils.go' tab is active and shows the following code:

```
utils > utils.go
1  package utils
2
3  func Helper() {
4      // какая-то полезная функция
5  }
```

Код в файле main.go



The image shows a screenshot of a Go IDE window. The window has a title bar with three small circles (macOS style). Below the title bar, there are two tabs: "utils.go" (with a Go logo and a red dot) and "main.go" (with a Go logo and a black dot). The "main.go" tab is active. The code in the editor is as follows:

```
main.go > ...
1  package main
2
3  import (
4      "myproject/utils"
5      "fmt"
6  )
7
8  func main() {
9      utils.Helper()
10     fmt.Println("Программа запущена")
11 }
```

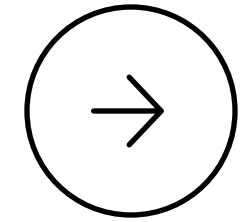
Связь пакетов и папок

- Go связывает пакеты с папками
- Папка `utils/` содержит пакет `utils`
- Папка `main/` содержит `main.go` и точку входа
- Главное: в каждой папке должен быть хотя бы один `.go`-файл с `package`

Почему такая структура удобна

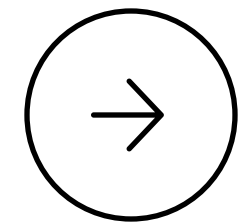
- Легко ориентироваться в проекте
- Можно переиспользовать код
- Удобно масштабировать проект
- Просто находить нужные файлы

Минимум для начала



Для первого проекта достаточно:

- `go.mod` — модуль
- `main.go` — основная логика



**Остальное — добавляется
по мере роста**

Итоги видео

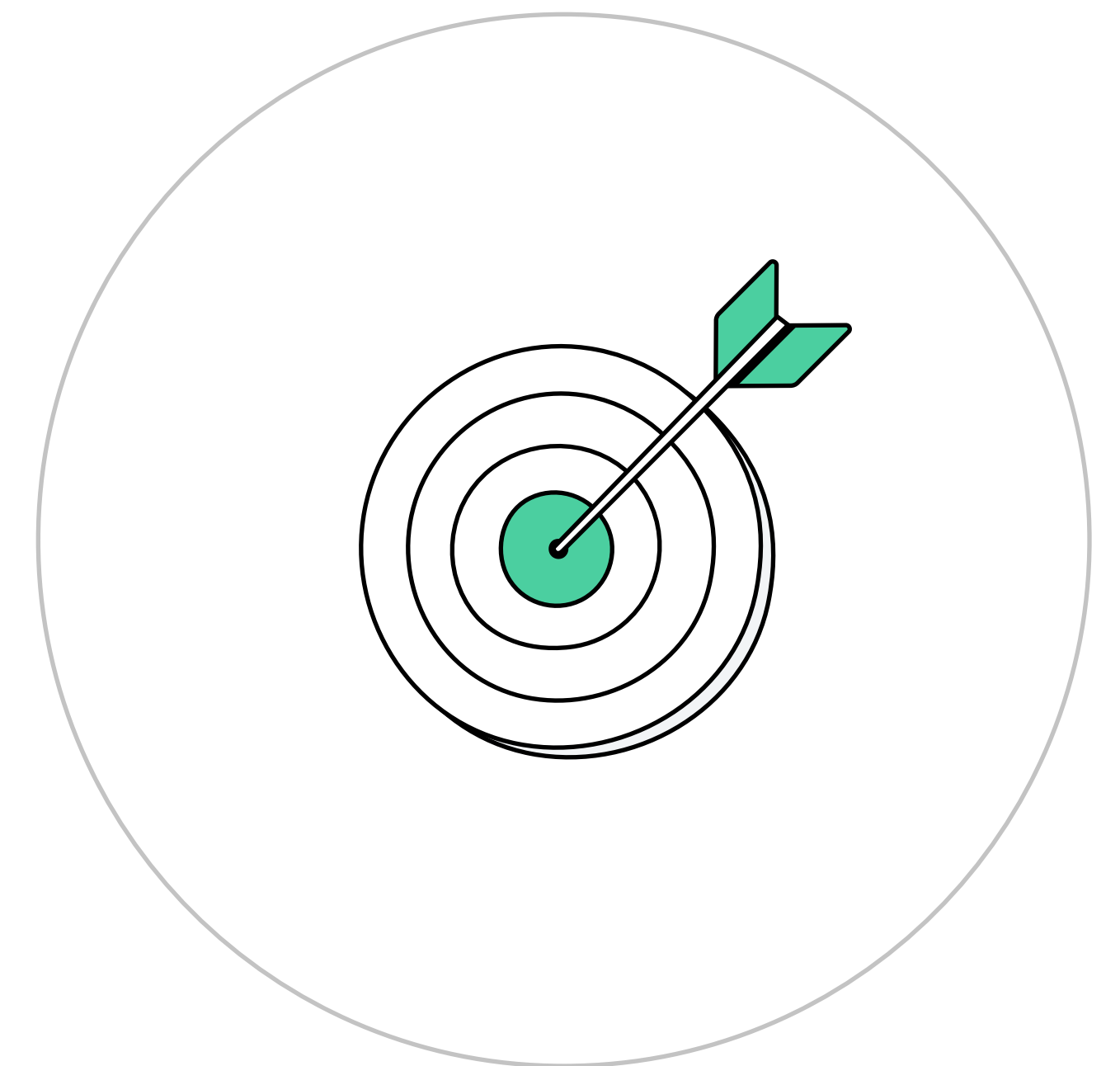
- Go-проект состоит из модуля и одного или нескольких пакетов
- Папки в проекте = пакеты. Внутри — .go-файлы с одинаковым package
- Файл go.mod — в корне, описывает модуль
- main.go — точка входа, лежит рядом с go.mod
- Структура Go-проекта простая и масштабируемая: легко добавлять новые пакеты и папки по мере роста кода



Подключение и использование своих пакетов

Цели видео

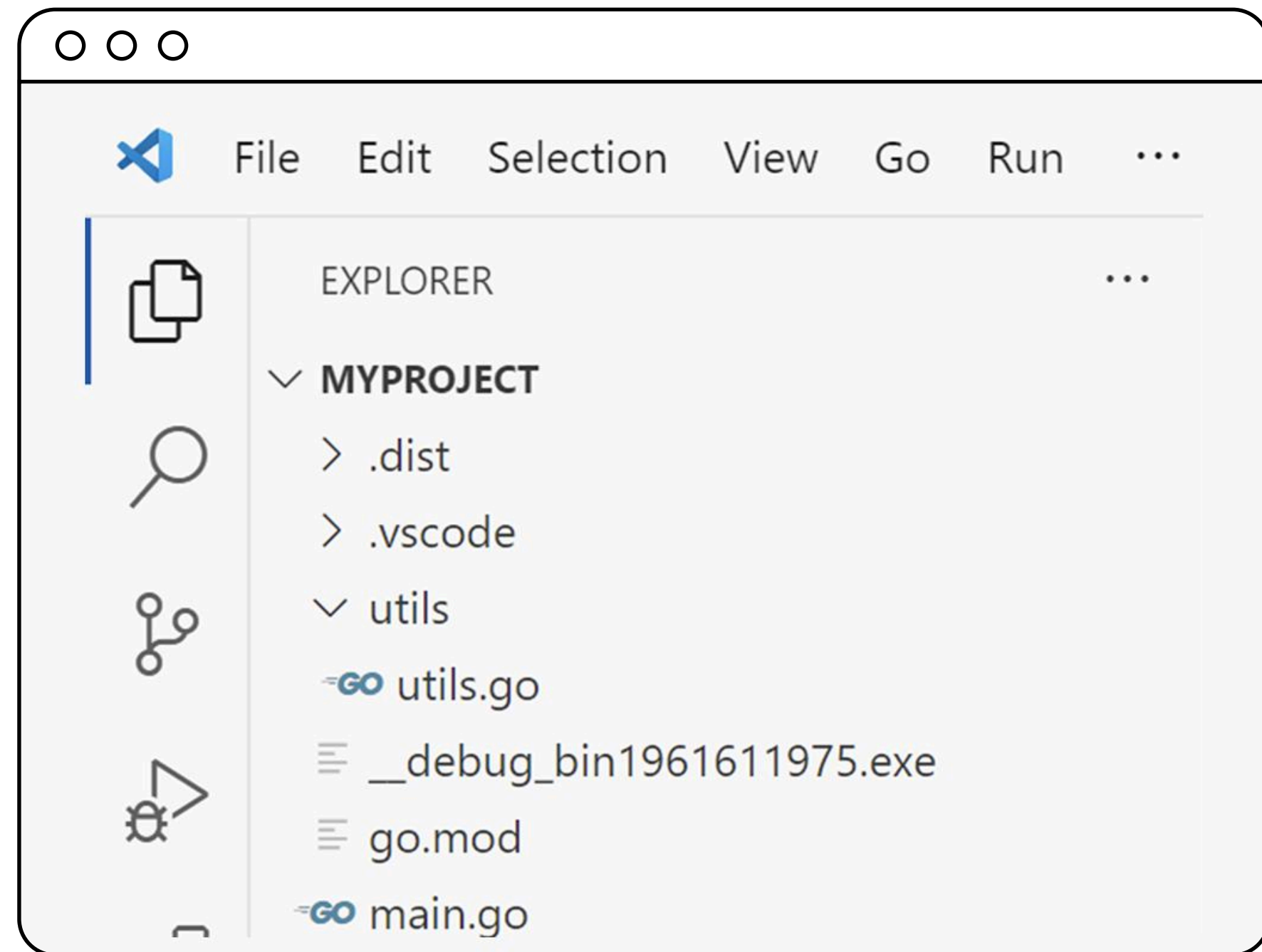
- Знать как подключать пользовательские пакеты в Go
- Понять, как правильно импортировать пакет в `main.go`
- Разобраться, как работает путь к пакету через имя модуля
- Узнать, какие функции доступны при импорте и почему важна заглавная буква
- Увидеть на практике, как использовать код из собственного пакета



Зачем использовать свои пакеты

- Выносите повторяющийся код в отдельные части
- Упрощаете структуру проекта
- Повышаете читаемость и повторное использование кода

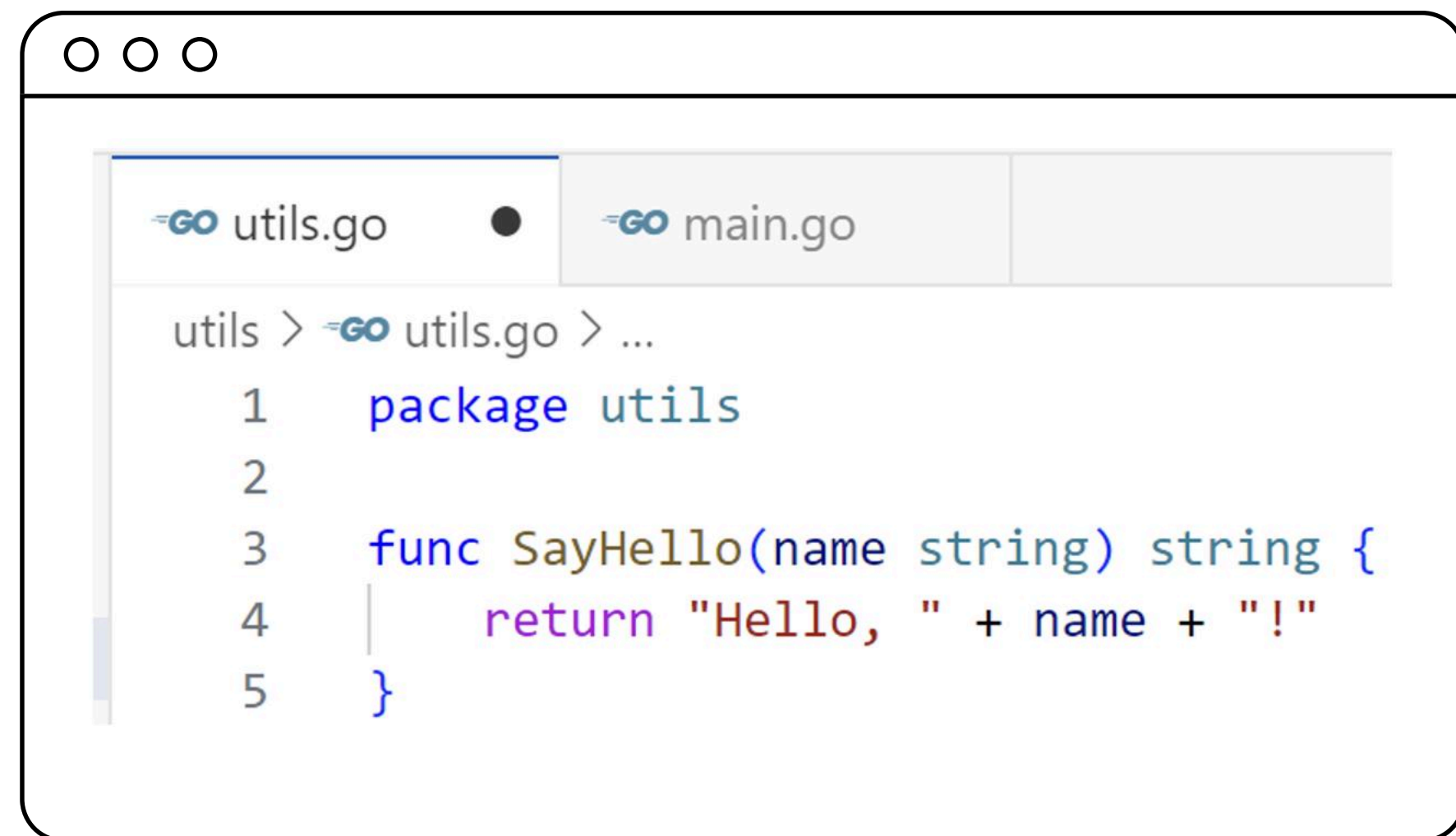
Структура проекта с пользовательским пакетом



- Папка `utils/` содержит пакет `utils`
- Файл `main.go` использует этот пакет

Структура проекта с пользовательским пакетом

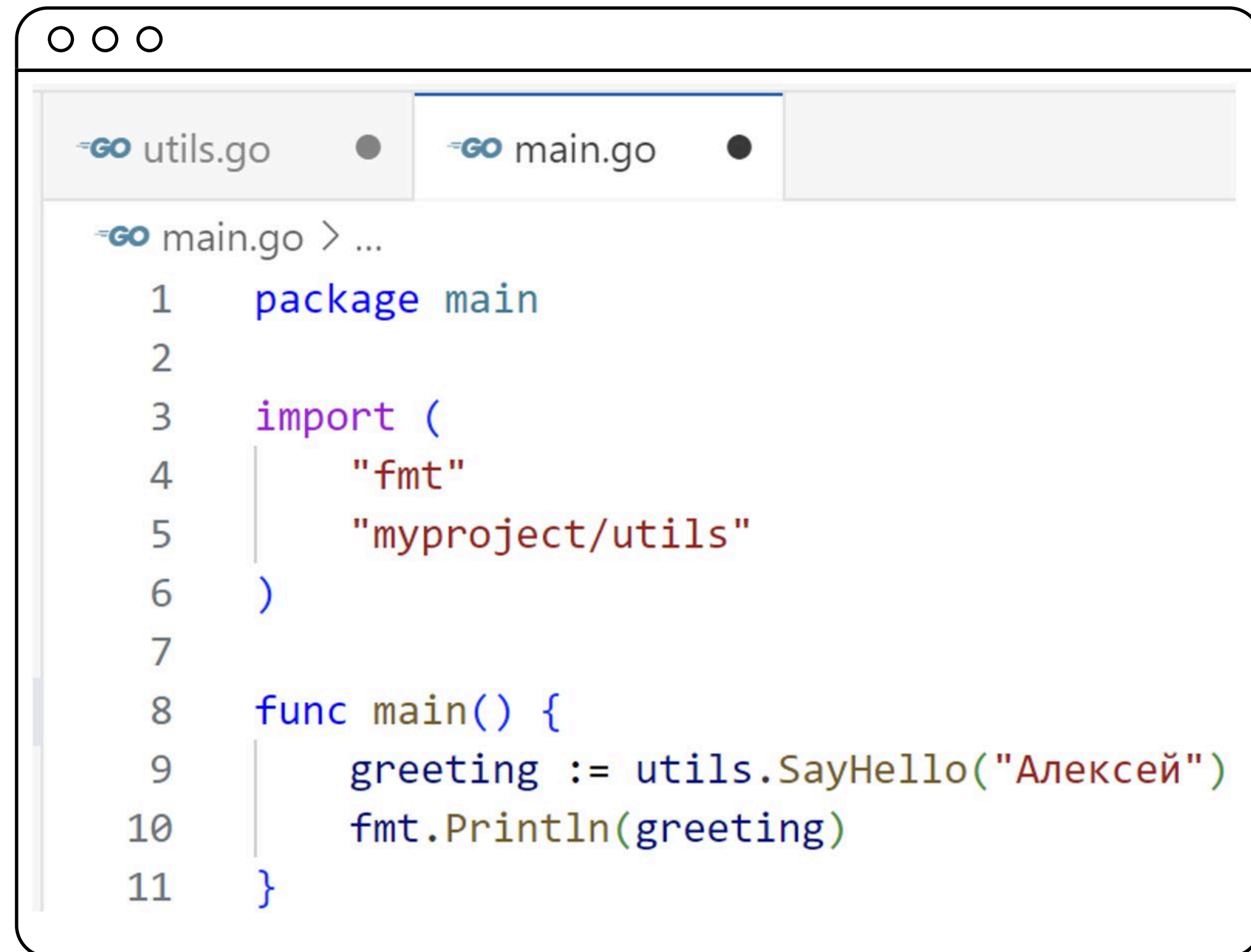
- Функция начинается с заглавной буквы — значит, она экспортируется
- Имя пакета совпадает с именем папки



The screenshot shows a Go IDE window with three tabs: `utils.go` (active), `main.go`, and an empty tab. The `utils.go` tab contains the following code:

```
utils > go utils.go > ...  
1  package utils  
2  
3  func SayHello(name string) string {  
4      return "Hello, " + name + "!"  
5  }
```


Использование пакета в main.go



```
GO utils.go • GO main.go •
GO main.go > ...
1  package main
2
3  import (
4      "fmt"
5      "myproject/utils"
6  )
7
8  func main() {
9      greeting := utils.SayHello("Алексей")
10     fmt.Println(greeting)
11 }
```

Импорт происходит по схеме:
имя_модуля/путь_к_пакету

Важно помнить

- Имя пакета = имя папки
- Все .go-файлы в папке должны использовать одно и то же имя пакета
- Функции, которые нужно использовать снаружи, должны начинаться с заглавной буквы
- Импорт происходит через имя_модуля/путь_к_папке

Как подключать свои пакеты

- Создайте отдельную папку
- Напишите в .go-файлах нужные функции
- Убедитесь, что они экспортируемые (с заглавной буквы)
- Импортируйте в main.go:

```
import "myproject/utls"
```


Итоги видео

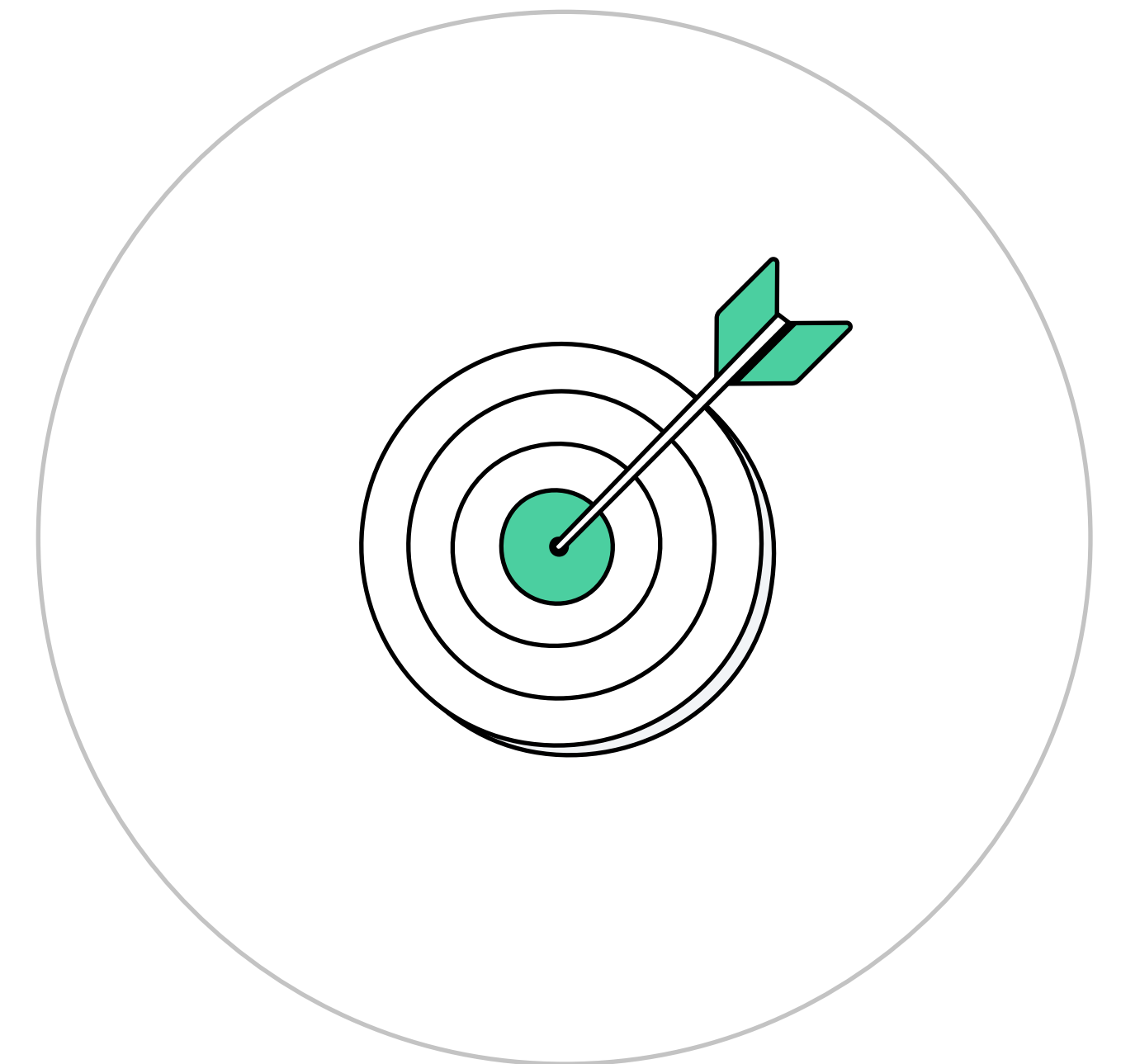
- Пользовательские пакеты хранятся в отдельных папках
- Имя пакета = имя папки, а все .go-файлы внутри используют одинаковое package
- Чтобы использовать функцию из пакета, она должна начинаться с заглавной буквы — быть экспортируемой
- Импорт производится по схеме: имя_модуля/путь_к_папке
- После импорта вы можете использовать функции через имя_пакета.Функция()



Чем отличается подход Go от других языков

Цели видео

- Понять ключевые отличия Go от Python, Java и JavaScript
- Узнать, почему в Go нет классов и как организована логика без ООП
- Разобраться, как работает система импортов в Go
- Увидеть, как просто запускать и собирать проекты на Go
- Оценить удобство управления зависимостями через `go.mod`



Go работает иначе, чем привычные языки

Если вы приходите в Go из Python, Java или JavaScript — вы сразу заметите отличия.

Go предлагает другой взгляд на организацию кода, сборку и зависимости

В Go нет классов

Вместо классов Go использует:

- ① Пакеты для группировки кода
- ② Функции и структуры
- ③ Методы, привязанные к типам
- **Классы и объектная модель отсутствуют**
- **Меньше абстракций – проще понимать и писать код**

Прозрачные импорты

- В каждом файле явно указаны все импорты
- Нет скрытых зависимостей
- Легко понять, откуда берётся каждый кусок кода

```
import (  
    "fmt"  
    "myproject/utls"  
)
```

Простая сборка без конфигураций

- Чтобы запустить Go-программу,
достаточно одной команды:

`go run main.go`

- Не нужны Maven, Gradle
или громоздкие конфиги
- Всё работает «из коробки»

Управление зависимостями — просто

- Используется файл `go.mod`
- Все зависимости и их версии фиксируются автоматически
- Сборка воспроизводима у всех членов команды

Без сторонних менеджеров, как в JavaScript или Python

В чём сила подхода Go

- Простой и читаемый код
- Минимум «магии»
- Быстрый старт без лишних настроек
- Удобная масштабируемость

Итоги видео

- В Go нет классов, вместо них — функции и методы, привязанные к типам
- Импорты в Go всегда явно указаны в начале файла — без скрытых зависимостей
- Go не требует сборщиков вроде Maven или Webpack — запуск прост: `go run main.go`
- Управление зависимостями встроено в язык через `go mod` и файл `go.mod`
- Подход Go — это простота, прозрачность и минимализм без лишней абстракции



Итоги



Итоги занятия

- Пакет — основная единица кода в Go, каждый .go-файл принадлежит пакету
- Модуль объединяет пакеты в единый проект и управляет зависимостями через go.mod
- Структура проекта в Go проста: корень с go.mod, папки — это пакеты
- Чтобы использовать свой пакет, нужно создать его в отдельной папке и импортировать по пути
- В Go нет классов, но есть простая и прозрачная модель организации кода без лишней сложности



Пакеты и модули в Go

Матвей Ефимов
Эксперт в Go, Python, SQL

